ELTE
FACULTY OF
INFORMATICS

PROGRAMMING
Lecture 3-4

Zsuzsa Pluhár
pluharzs@inf.elte.hu

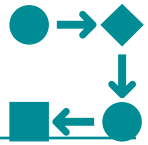
Refreshing

Specification

Components:

- **Input data** (identifier, set of values [unit of measure])
- Information about the input (**precondition**)
- **Output**, results (identifier, set of values)
- The statement – true for all input and output – used to get the result (**postcondition**)
- Definitions of the used concepts
- Requirements against the solution
- Restricting conditions

The algorithm



Elements of the algorithms:

- Sequence (step by step execution)
- Conditional branch (choice from 2 or more options based on a condition)
- Loop (repeat certain amount of times or until a certain condition is met)
- Subprogram (a complex activity, with a unique name – abstraction)

Programming on **analogous* way**

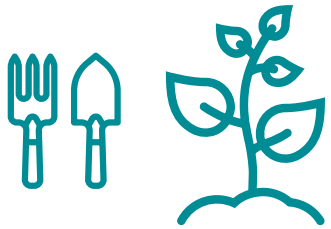
* similar

Problem solving in analogous way

- When you need to solve a problem - earlier you solved a similar problem → you can solve it based on the previous solved problem.
- General problem-solving method:
  - practical experience, learned knowledge
  - in all filed, domains of life
  - also in the toolbox of a practising programmer

Problem solving in analogous way

- Examples
  - planting,
  - cooking,
  - assembling,
  - …



Problem solving in analogous way

- **Aim**: create a program (algortihm) to solving a problem

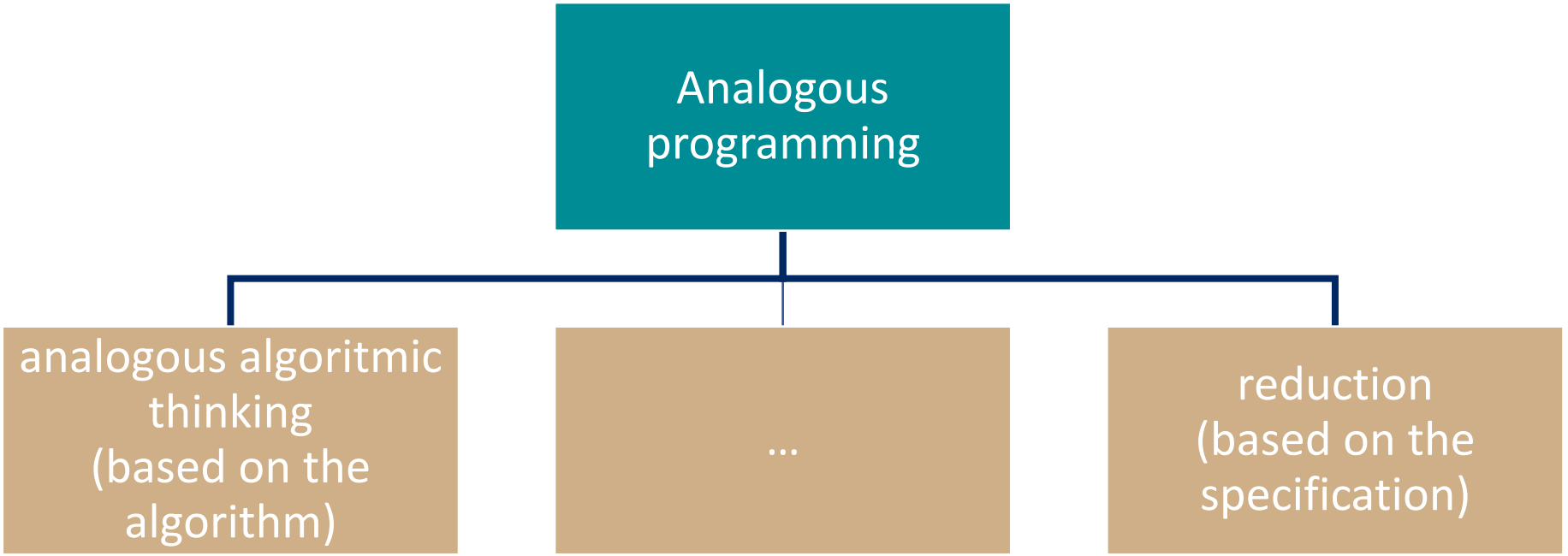
every time find out the solution

or

find a similar problem and use the solution as a basis

Programming in analogous way: the solution of a specific problem is based on the solution to a previous problem.

Problem solving in analogous way



Analogous programming – algorithmic thinking

"sample problem" **"specific problem"**

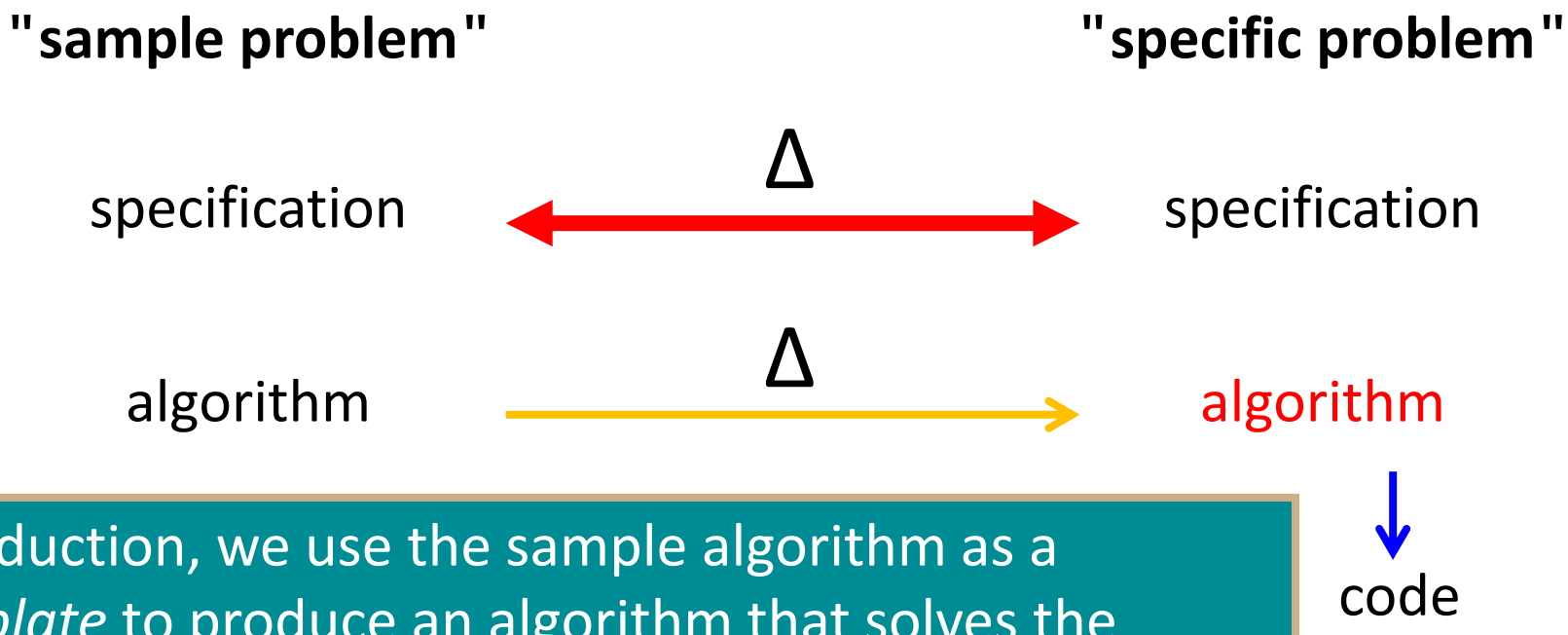
specification  specification

algorithm  algorithm

code

Problems similar to the sample problem are solved by using, copying the ideas from the sample task, using analogous algorithmic thinking.

Analogous programming – reduction



In reduction, we use the sample algorithm as a *template* to produce an algorithm that solves the specific problem, checking the differences between the specification of the specific and the sample problem.

Analogous programming – reduction

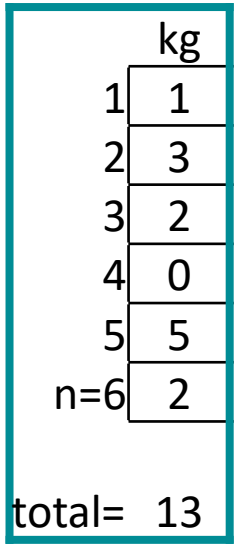
- In reduction, we use the algorithm of the sample problem as a **template** to produce an algorithm that solves the specific problem, checking the **differences** between the **specification** of the specific and the sample problem.
- If the correctness of the template (pattern) is proofed - the reduction don't change the correctness -> the specific problem will be solved correct…

Example problem

- A paper collection is organised in a secondary school. We know how many kilos of paper each student collected. Find out how much paper was collected in total?

example:
[1, 3, 2, 0, 5, 2] → 13

example:
n=6, kg=[1, 3, 2, 0, 5, 2] → total=13



Example problem

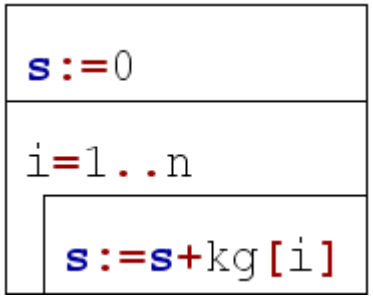
Task: calculate the sum of the elements of an array with n numbers

example:
n=6, kg=[1, 3, 2, 0, 5, 2] → total=13

Specification:

```
In: n∈N, kg∈N[1..n]
Out: s∈N
Pre: -
Post: s = SUM(i=1..n, kg[i])
```

Algorithm:



$$s = \sum_{i=1}^{n} kg[i]$$

	kg
1	1
2	3
3	2
4	0
5	5
n=6	2
total=	13

Example$_2$ – scalar product

Task:

Calculate the scalar product of two vectors with the same dimension

example:
n=3,
x=[1, -3, 2], → s = -9
y=[4, 5, 1]

	x	y	x[i]*y[i]
1	1	4	4
2	-3	5	-15
n=3	2	1	2
		s=	-9

Example$_2$ – scalar product

Task:

Calculate the scalar product of two vectors with the same dimension

	x	y	x[i]*y[i]
1	1	4	4
2	-3	5	-15
n=3	2	1	2
		s=	-9

Specification:

```
In: n∈N, x∈Z[1..n], y∈Z[1..n]
Out: s∈Z
Pre: -
Post: s = SUM(i=1..n, x[i]*y[i])
```

$$s = \sum_{i=1}^{n} x[i] * y[i]$$

Example$_2$ – scalar product

collecting paper:

```
In: n∈N, kg∈N[1..n]
Out: s∈Z
Pre: -
Post: s = SUM(i=1..n, kg[i])
```

scalar product:

```
In: n∈N, x∈Z[1..n], y∈Z[1..n]
Out: s∈Z
Pre: -
Post: s = SUM(i=1..n, x[i]*y[i])
```

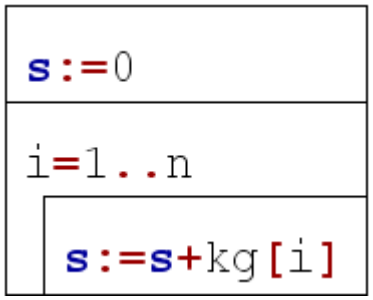
Comparing the specification → the 2 problems are similar

Let's create the algorithm using
- analogous algorithmic thinking
- reduction

Example$_2$ – scalar product

analogous algorithmic thinking

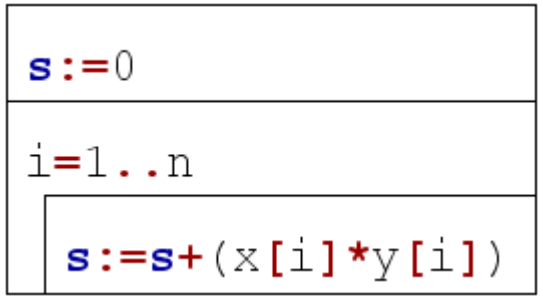
collecting paper:



During the solution we calculate the sum continuously.

We use an integer variable (i) to index all of the elements in the array and add the kg[i] referenced value to the sum (started with a 0 value) => The algorithm is a value assignment, then in a loop from 1 to n.

scalar product:



During the solution we calculate the sum continuously.

We use an integer variable (i) to index all of the elements in the arrays and add the x[i]*y[i] referenced value to the sum (started with a 0 value) => The algorithm is a value assignment, then in a loop from 1 to n.

Example$_2$ – scalar product
reduction

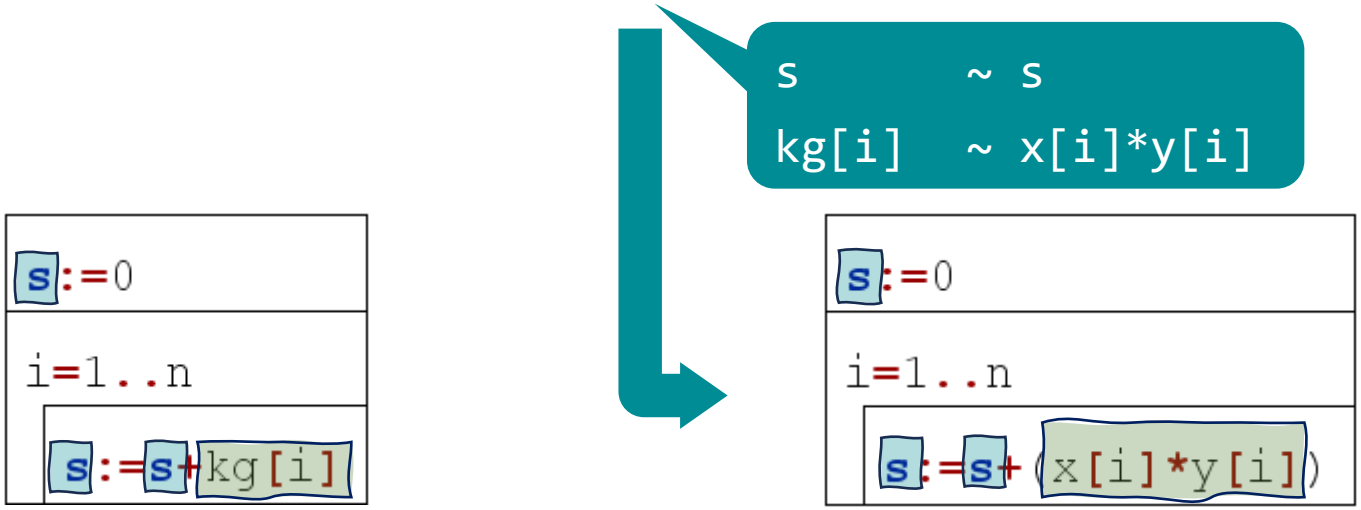
collecting paper:

```
In: n∈N, kg∈N[1..n]
Out: s∈Z
Pre: -
Post: s = SUM(i=1..n, kg[i])
```

scalar product:

```
In: n∈N, x∈Z[1..n], y∈Z[1..n]
Out: s∈Z
Pre: -
Post: s = SUM(i=1..n, x[i]*y[i])
```

Algorithm



Example$_3$ – sum of numbers

example:
n=1, m=5 → s=15
n=-2, m=2 → s=0

Task:

Calculate the sum all of the integers between the two integers a and b (a≤b)!

Specification:

```
In: n∈Z, m∈Z
Out: s∈Z
Pre: n<=m
Post: s = SUM(i=n..m, i)
```

$$s = \sum_{i=n}^{m} i$$

n=	-2
	-1
	0
	1
m=	2
s=	0

Example$_3$ – sum of numbers

collecting paper:

```
In: n∈N, kg∈N[1..n]
Out: s∈Z
Pre: -
Post: s = SUM(i=1..n, kg[i])
```

sum of numbers:

```
In: n∈Z, m∈Z
Out: s∈Z
Pre: n<=m
Post: s = SUM(i=n..m, i)
```

Comparing the specification → the 2 problems are similar

Let's create the algorithm using
- ~~analogous algorithmic thinking~~
- reduction

Example$_3$ – sum of numbers
reduction

collecting paper:

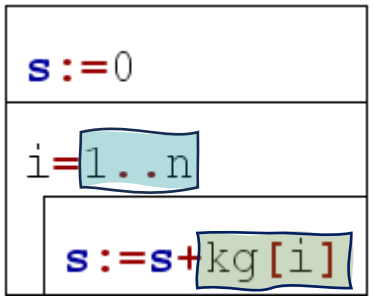
```
In: n∈N, kg∈N[1..n]
Out: s∈Z
Pre: -
Post: s = SUM(i=1..n, kg[i])
```

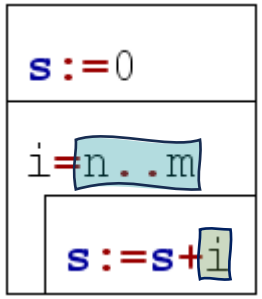
sum of numbers:

```
In: n∈Z, m∈Z
Out: s∈Z
Pre: a<=b
Post: s = SUM(i=n..m, i)
```

```
s      ~ s
1..n   ~ n..m
kg[i]  ~ i
```

Algorithm





Conclusion

- The problems (tasks) are similar – they can be solved using a template
- How can they be classified under the same template?
- How could such tasks be formulated in general?

Similarities

closed interval of numbers - loop

collecting paper

```
s=SUM(i=1..n, kg[i])
```

	kg
1	1
2	3
3	2
4	0
5	5
6	2

	i		kg[i]
b=	1	→	1
	2	→	3
	3	→	2
	4	→	0
	5	→	5
e=	6	→	2

index interval in an array

scalar product

```
s=SUM(i=1..n, x[i]*y[i])
```

	x	y
1	1	4
2	-3	5
3	2	1

	i		x[i]*y[i]
b=	1	→	4
	2	→	-15
e=	3	→	2

index interval in an array

Defines the intervall of the loop variable (counter)

sum of numbers

```
s=SUM(i=n..m, i)
```

	i		i
b=	-2	→	1
	-1	→	3
	0	→	2
	1	→	0
e=	2	→	5

closed interval

Similarities

assigned value to the elements of an interval

collecting paper

```
s=SUM(i=1..n, kg[i])
```

	kg
1	1
2	3
3	2
4	0
5	5
6	2

	i		kg[i]
b=	1	→	1
	2	→	3
	3	→	2
	4	→	0
	5	→	5
e=	6	→	2

element of an array

scalar product

```
s=SUM(i=1..n, a[i]*b[i])
```

	x	y
1	1	4
2	-3	5
3	2	1

	i		x[i]*y[i]
b=	1	→	4
	2	→	-15
e=	3	→	2

product of values of two arrays

Function (assignment) that assigns a value to the elements of the interval: i → f(i)

sum of numbers

```
s=SUM(i=n..m, i)
```

	i		i
b=	-2	→	1
	-1	→	3
	0	→	2
	1	→	0
e=	2	→	5

the ith element in the interval

Similarities
summation

collecting paper

```
s=SUM(i=1..n, kg[i])
```

	kg		i		kg[i]
1	1	b=	1	→	1
2	3		2	→	3
3	2		3	→	2
4	0		4	→	0
5	5		5	→	5
6	2	e=	6	→	2

s

scalar product

```
s=SUM(i=1..n, a[i]*b[i])
```

	x	y		i		x[i]*y[i]
1	1	4	b=	1	→	4
2	-3	5		2	→	-15
3	2	1	e=	3	→	2

s

sum of numbers

```
s=SUM(i=n..m, i)
```

	i		i
b=	-2	→	1
	-1	→	3
	0	→	2
	1	→	0
e=	2	→	5

s

The assigned value need to be added together from 0: s=0+f(e)+f(e+1)+…+f(u)

Problem in general

Let an interval of integers [b..e] and a function f: [b..e]→H be given. (Let us suppose that the addition operation is defined on the elements of H).

Determine the sum of the values of the f function on the interval [b..e], i.e. the value of the expression f(b)+f(b+1)+...+f(e)

i		f(i)
b	→	f(b)
b+1	→	f(b+1)
...	→	...
e-1	→	f(e-1)
e	→	f(e)

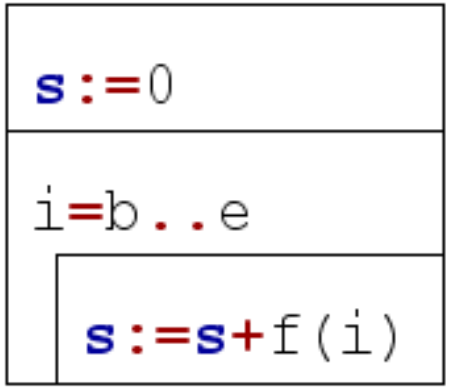
Problem in general

Specification:

```
In: b∈Z, e∈Z
Out: s∈H
Pre: -
Post: s=SUM(i=b..e, f(i))
```

i		f(i)
b	→	1
b+1	→	3
…	→	2
e-1	→	0
e	→	5

Algorithm:



During the solution we calculate the sum continuously.

We use an integer variable (i) to index all of the **elements in the interval** and add the **f(i)** referenced value to the **sum** (started with a 0 value) => The algorithm is a value assignment, then in a loop from b to e.

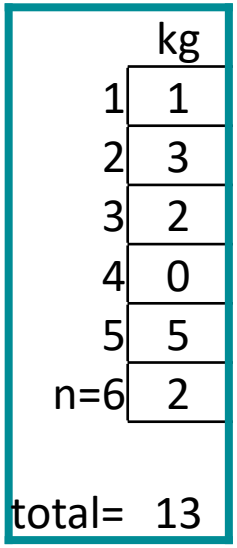
Example problem

Task: calculate the sum of the elements of an array with n numbers

example:
n=6, kg=[1, 3, 2, 0, 5, 2] → total=13

Specification:

```
In: n∈N, kg∈N[1..n]
Out: s∈N
Pre: -
Post: s = SUM(i=1..n, kg[i])
```



Check the differences betwwen the 2 problems and replace the relevant parts

Example – with reduction

Pattern:

```
In: b∈Z, e∈Z
Out: s∈H
Pre: -
Post: s=SUM(i=b..e, f(i))
```

collecting paper:

```
In: n∈N, kg∈N[1..n]
Out: s∈Z
Pre: -
Post: s = SUM(i=1..n, kg[i])
```

Reduction:

s	~	s
b..e	~	1..n
f(i)	~	kg[i]

Algorithm:

Programming patterns

Kind of problems

- Program can be created ad-hoc
- There are strategies, solutions that can solve very similar problems – with small modifications
- Kind of problems –> templates
- Let create a CORRECT solution (template)
- Analogous programming: we use those templates as **PATTERN**s

Programming pattern

Its **aim:**

- A proven template as a basis, on which we can build our solutions later. (Development will be quicker and safer)

Its **structure**:

1. abstract task specification [+program specification]
2. abstract algorithm

Comment: the input is at least a sequence…

Programming pattern (PoA)

How we use them:

1. create a specification for the given, specific task
2. guess the PoA from specification
3. "map" the parameters of the given task to the parameters of the corresponding abstract task = reduction
4. "generate" the task-specific algorithm based on the algorithm of the PoA, by changing the parameters as per point 3.
5. create a more efficient algorithm using program transformations

Programming patterns
Summation

Summation

Tasks (examples):

1. We know the monthly income and expenses of a person. Let's calculate by **how much** money his asset will change by the end of the year!
2. We know the lap times of a car racer. Let's determine his **average** lap time!
3. Let's calculate the **factorial** of N!
4. We know N words. Give the sentence we get by **joining** them.

Summation

Tasks (examples):

1. We know the monthly income and expenses of a person. Let's calculate by **how much** money his asset will change by the end [illegible] ar!
2. We know the [illegible] racer. Let's determine
3. Let's calcula[illegible].
4. We know N [illegible]. Give the sentence we get by **joining** them.

What is common?
Calculate the sum of n numbers

Summation

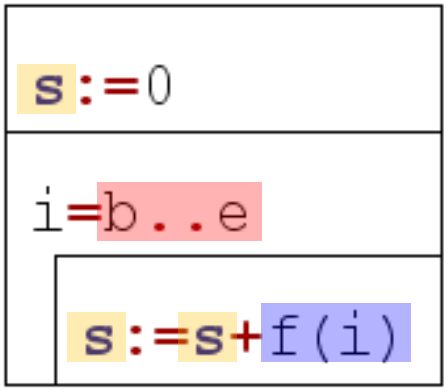
Let an interval of integers [b..e] and a function f: [b..e]→H be given. Let us suppose that the addition operation is defined on the elements of H.

Determine the sum of the values of the f function on the interval [b..e], i.e. the value of the expression $\sum_{i=b}^{e} f(i)$ (if b>e the value will be 0)

Specification:

```
In: b∈Z, e∈Z
Out: s∈H
Pre: -
Post: s=SUM(i=b..e, f(i))
```

Algorithm:



Example

We know the monthly income and expenses of a person. Let's calculate by how much money his asset will change by the end of the year!

Pattern:

```
In: b∈Z, e∈Z

Out: s∈H
Pre: -
Post: s=SUM(i=b..e, f(i))
```

Specific problem:

```
In: n∈N, m∈Tass[1..n]
Tass=In x Exp, In=N, Exp=N
Out: s∈Z
Pre: -
Post: s = SUM(i=1..n,
              m[i].in-m[i].exp)
```

Reduction:

s	~	s
b..e	~	1..n
f(i)	~	m[i].in-m[i].exp

Example

We know the monthly income and expenses of a person. Let's calculate by how much money his asset will change by the end of the year!

Pattern:

```
In: b∈Z, e∈Z

Out: s∈H
Pre: -
Post: s=SUM(i=b..e, f(i))
```

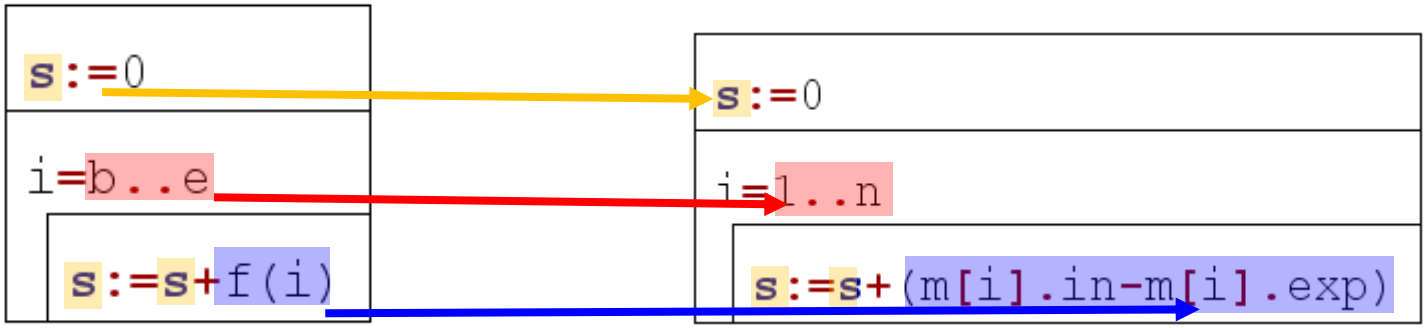
Specific problem:

```
In: n∈N, m∈Tass[1..n]
Tass=In x Exp, In=N, Exp=N
Out: s∈Z
Pre: -
Post: s = SUM(i=1..n, m[i].in-m[i].exp)
```

Reduction:

s	~	s
b..e	~	1..n
f(i)	~	m[i].in-m[i].exp

Algorithm:



Programming patterns
Counting

Counting

Tasks (examples):

1. We know the monthly income and expenses of a person. Let's count **the number of months** where his assets grow!
2. Let's calculate the **number of** divisors of an integer!
3. Let's determine **how many** letter "a" can be found in the name of a person!
4. Based on the annual daily statistics, let's count the **number of** days when frozen!
5. We have birth date (month) data from N people. Let's count **how many** of them have birthday during the winter!

Counting

Tasks (examples):

1. We know the monthly income and expens[illegible] of a person. Let's count **the number of mo[illegible]** his assets grow!
2. Let's calculate the **nu[illegible]**
3. Let's deter[illegible] in the[illegible]
4. B[illegible] count the **nu[illegible]**
5. We[illegible]ith) data from N people. Let's cou[illegible] of them have birthday during the wint[illegible]

What is common?

We have a sequence of "somethings", and we have to count how many of their items have a given attribute.

Counting – example

algorithmic thinking

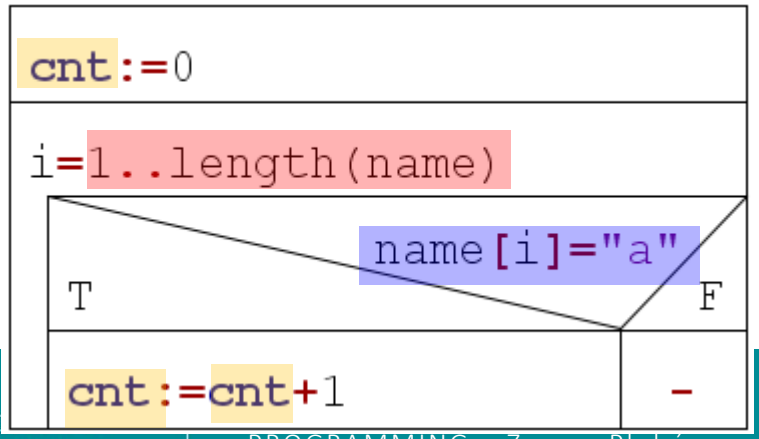
Task: Let's determine how many letter "a" can be found in the name of a person!

i	name	name[i]="a"	value
1	a	true	1
2	r	false	0
3	i	false	0
4	a	true	1
		cnt=	2

Specification:

```
In: name∈S
Out: cnt∈N
Pre: -
Post: cnt=SUM(i=1..length(name),1,name[i]="a")
```

Algorithm:



Counting

count how many of their items have a given attribute

Let an interval of integers [b..e] and a conditon A: [b..e]→L be given.

Determine the number of times A attains the "true" value on the interval [b..e] (in the case of an empty interval, if b>e the value returned will be 0)

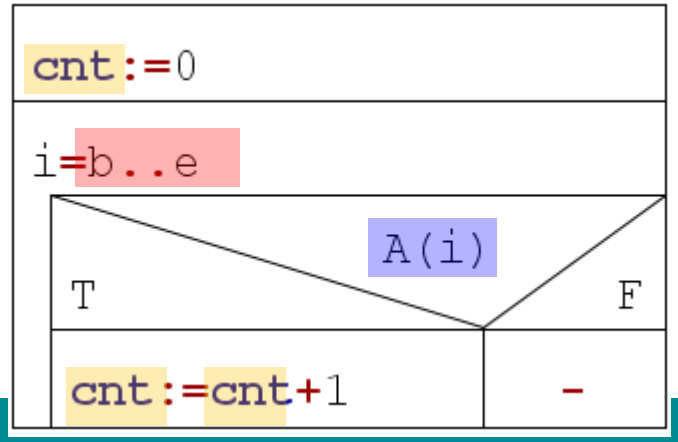
Specification:

```
In: b∈Z, e∈Z
Out: cnt∈N
Pre: -
Post: cnt=SUM(i=b..e, 1,A(i))
```

shorting:

```
Post: cnt=COUNT(i=b..e,A(i))
```

Algorithm:



Example

We have birth date (month) data from N people. Let's count how many of them have birthday during the winter!

Pattern:

```
In: b∈Z, e∈Z
Out: cnt∈N
Pre: -
Post: cnt=COUNT(i=b..e, f(i))
```

Specific problem:

```
In: n∈N, m∈N[1..n]
Out: cnt∈Z
Pre: ∀i∈[1..n]:(1<=m[i]<=12)
Post: cnt = COUNT(i=1..n,
                  m[i]<3 or m[i]=12)
```

Reduction:

cnt	~	cnt
b..e	~	1..n
A(i)	~	m[i]<3 or m[i]=12

the precondion of the specific problem can be stricter than in the used pattern

Example

We have birth date (month) data from N people. Let's count how many of them have birthday during the winter!

Pattern:

In: b∈Z, e∈Z
Out: cnt∈N
Pre: -
Post: cnt=COUNT(i=b..e, f(i))

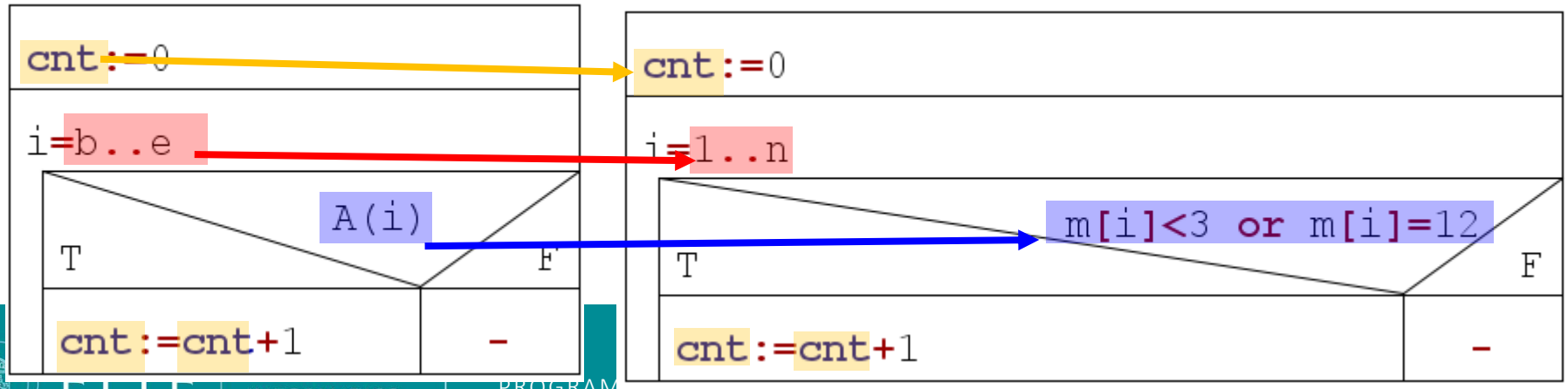
Specific problem:

In: n∈N, m∈N[1..n]
Out: cnt∈Z
Pre: ∀i∈[1..n]:(1<=m[i]<=12)
Post: cnt = COUNT(i=1..n, m[i]<3 or m[i]=12)

Reduction:

cnt	~	cnt
b..e	~	1..n
A(i)	~	m[i]<3 or m[i]=12

Algorithm



Example

We have birth date (month) data from N people. Let's count how many of them have birthday during the winter!

Pattern:

In: b∈Z, e∈Z
Out: cnt∈N
Pre: -
Post: cnt=COUNT(i=b..e, A(i))

Specific problem:

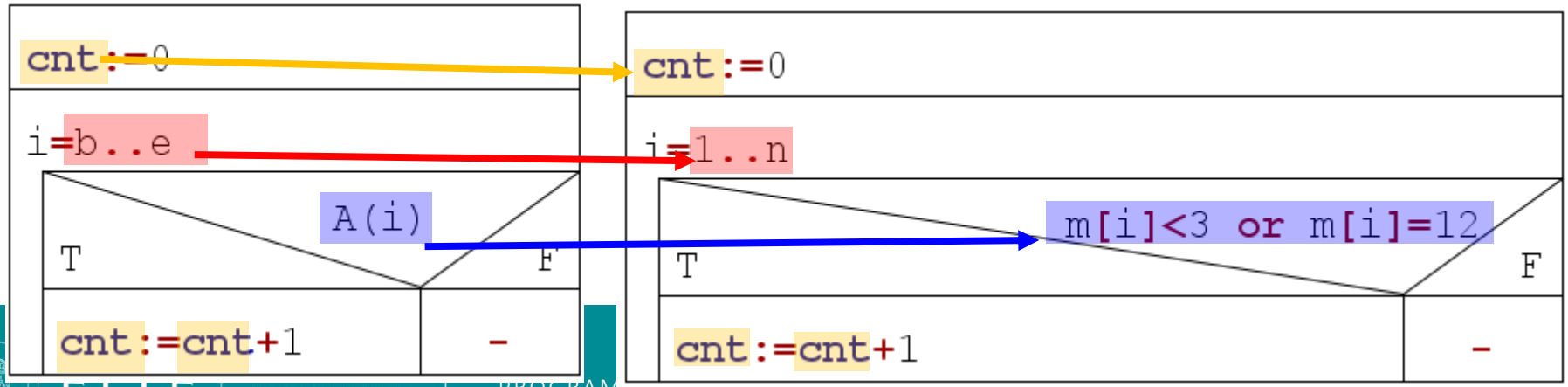
In: n∈N, m∈N[1..n]
Out: cnt∈Z
Pre: ∀i∈[1..n]:(1<=m[i]<=12)

What would happen if the precondition was not met?

Reduction:

cnt	~	cnt
b..e	~	1..n
A(i)	~	m[i]<3 or m[i]=12

Algorithm



Programming patterns
Maximum selection

Maximum selection

Tasks (examples):

1. We know the monthly income and expenses of a person. Let's determine the month where his assets grow the **most**!
2. Let's pick the name of the person who is the **last** in the alphabetical order!
3. Let's find the person, who likes the **most** types of food!
4. Based on the annual daily statistics, let's define the **warmest** day during the year!
5. We have birth dates from N people, let's find who has birthday in the year **first**!

Maximum selection

Tasks (examples):

1. We k…

What is common?

We have to pick/find the greatest (or least) something from a sequence of "somethings".

Important:
The somethings have an attribute based on which we can sort them (sorting relation).
If we have at least 1 element, then we know that there exists a maximum (or minimum).

4. …s, let's define the …

5. …es from N people, let's find who has … the year **first**!

Maximum selection – example

algorithmic thinking

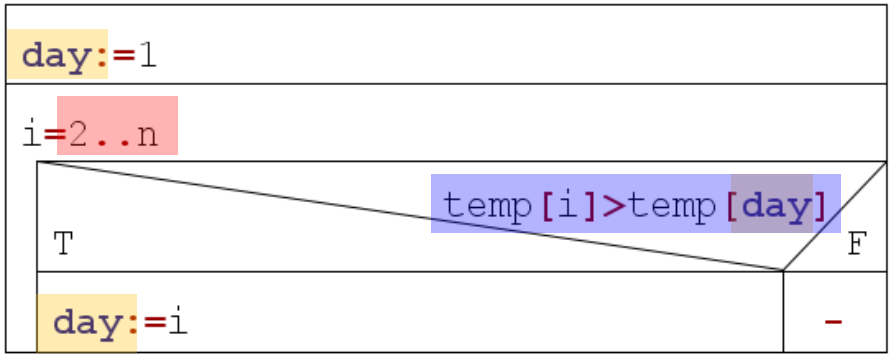
Task: Based on the annual daily statistics, let's define the **warmest** day during the year!

i	temp	day (i=1)	day (i=2)	day (i=3)	day (i=4)
1	13,5	13,5	13,5	13,5	13,5
2	12,6	12,6	12,6	12,6	12,6
3	14,8	14,8	14,8	14,8	14,8
4	10,2	10,2	10,2	10,2	10,2

Specification:

```
In: n∈N, temp∈R[1..n]
Out: day∈N
Pre: n>0 and ∀i∈[1..n]:(-100<=temp[i]<=100)
Post: day∈[1..n] and ∀i∈[1..n]:(temp[nap]>=temp[i])
```

Algorithm:



Maximum selection

Let a non-empty interval of integers [b..e] and a function f: [b..e]→H be given. Let us suppose that a total order is defined on the elements of H.

Determine the point (index) at which attains its maximum value on the interval [b..e] and calculate this **value** too. (if there is more than one maximum, any of them can be selected)

Specification:

```
In: b∈Z, e∈Z
Out: maxind∈Z, maxval∈H
Pre: b<=e
Post: maxind∈[e..u] and
    ∀i∈[b..e]:(f(maxind)>=f(i)) and
    maxval=f(maxind)
```

Maximum selection

Let a non-empty interval of integers [b..e] and a function f: [b..e]→H be given. Let us suppose that a total order is defined on the elements of H.

Determine the point (index) at which attains its maximum value on the interval [b..e] and calculate this **value** too. (if there is more than one maximum, any of them can be selected)

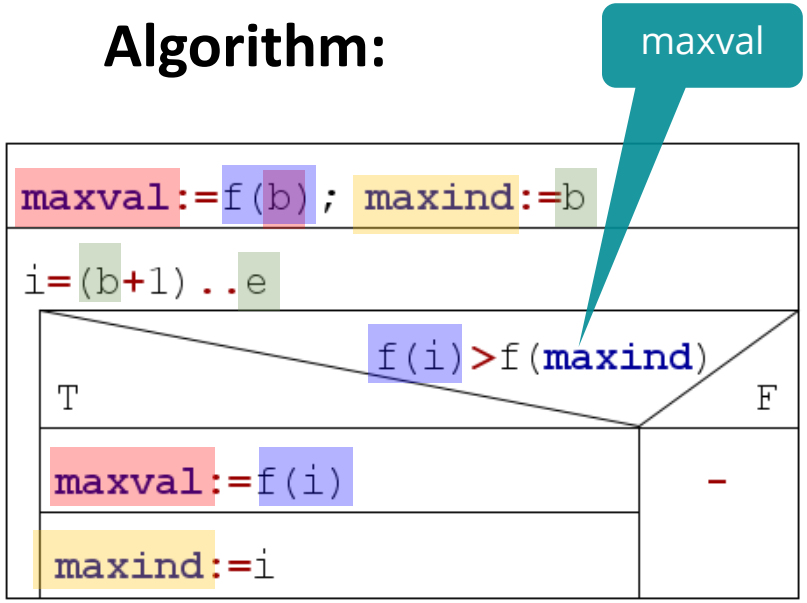
Specification:

```
In: b∈Z, e∈Z
Out: maxind∈Z, maxval∈H
Pre: b<=e
Post: maxind∈[b..e] and
     ∀i∈[b..e]:(f(maxind)>=f(i)) and
     maxval=f(maxind)
```

shorting:

```
Post:
(maxind,maxval)=MAX(i=b..e,f(i))
```

Algorithm:



Example

Let's pick the name of the person who is the last in the alphabetical order!

Pattern:

In: b∈Z, e∈Z

Out: maxind∈Z, maxval∈H

Pre: b<=e

Post: (maxind,maxval)=

MAX(i=b..e,f(i))

Specific problem:

In: n∈N, names∈S[1..n]

Out: lastind ∈N, lastn∈S

Pre: n>0

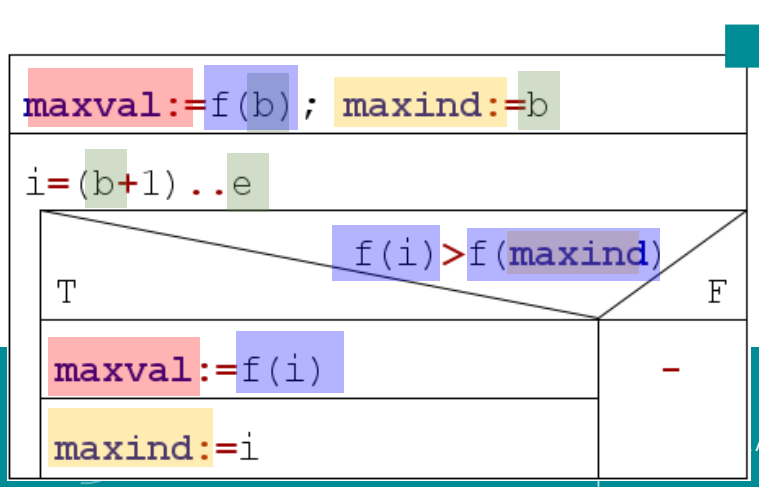
Post: (lastind,lastn)=

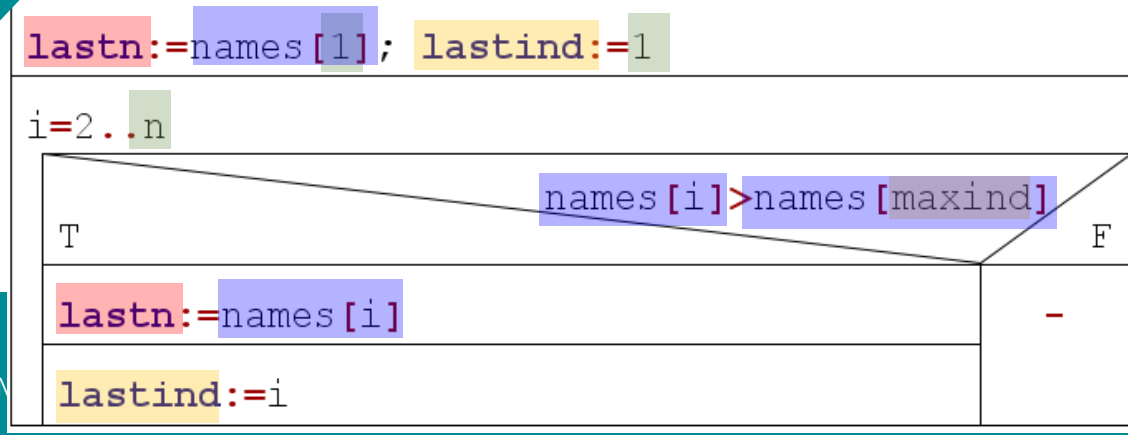
MAX(i=1..n,names[i])

Reduction:

maxind, maxval	~	lastind, lastn
b..e	~	1..n
f(i)	~	names[i]

Algorithm





Programming patterns
Search

Search

Tasks (examples):

1. We know the monthly incomes and expenses of a person. His assets grow by the end of the year. Let's **give a** month, when his assets didn't grow!
2. Let's **define a** non-1 and non-N divisor of the integer N!
3. Let's **search for** letter "a" in the name of a person!
4. Let's **search for** a course in which the student failed!
5. Let's **find an** element in a sequence that is greater than the previous element!

Search

Tasks (examples):

1. We know the monthly in… f a person. His assets… t's **give a** mon…
2. …r
3. …
4. …student failed!
5. L… a sequence that is greater th… element!

What is common?

We have a sequence of "somethings", and we have to search for an element that has a given attribute, and we do not know whether such an element exists in the sequence.

Search – example

algorithmic thinking

Task: Let's **define a** non-1 and non-N divisor of the integer N!

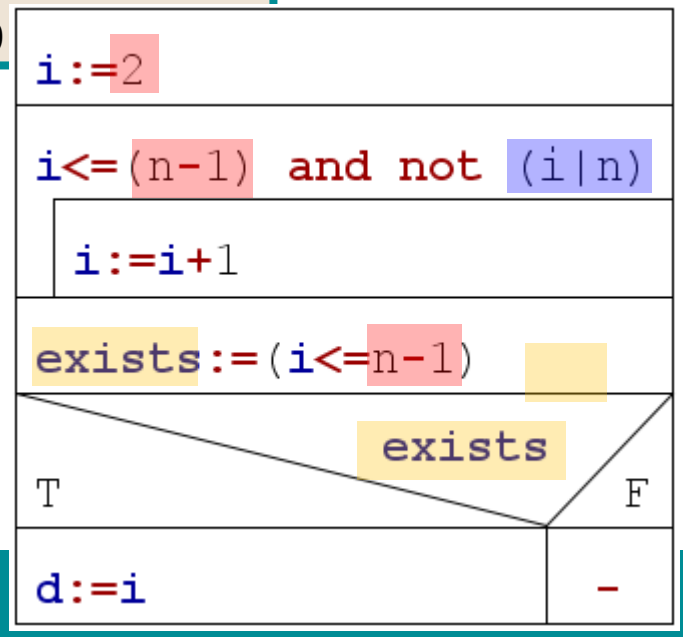
Specification:

```
In: n∈N
Out: d∈N, exists∈L
Pre: n>1
Post:
exists=∃i∈[2..n-1]:(i|n) and
exists->(d∈[2..n-1] and (d|n) and
    ∀i∈[2..d-1]:(i∤n)
```

	i	i<=8?	i ∤ n?
	1		
b=	2	true	true
	3	true	false
	4		
	5		
	6		
	7		
e=	8		
n=	9		

	i	i<=4?	i ∤ n?
	1		
b=	2	true	true
	3	true	true
e=	4	true	true
n=	5	false	

Algorithm:



Search

Let an interval of integers [b..e] and a condition A: [b..e]→L be given.

Determine the ***first*** point (index) starting from left at which A attains the "true" value on the interval [b..e] if it exists. (The empty interval is an equivalent of A evaluating to "false" on the whole interval.)

Specification:

```
In: b∈Z, e∈Z
Out: exists∈L, ind∈Z
Pre: -
Post: exists=∃i∈[b..e]:(A(i)) and
        exists->(ind∈[b..e] and A(ind) and
           ∀i∈[b..ind-1]:(not A(i)))
```

Search

Let an interval of integers [b..e] and a condition A: [b..e]→L be given.

Determine the ***first*** point (index) starting from left at which A attains the "true" value on the interval [b..e] if it exists. (The empty interval is an equivalent of A evaluating to "false" on the whole interval.)

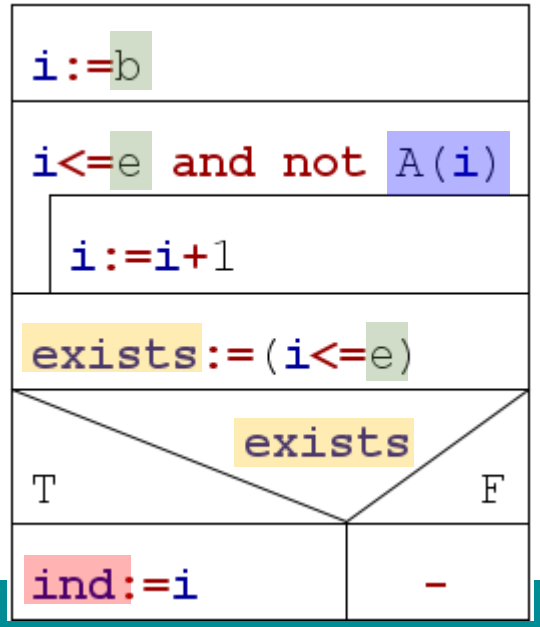
Specification:

```
In: b∈Z, e∈Z
Out: exists∈L, ind∈Z
Pre: -
Post: exists=∃i∈[b..e]:(A(i)) and
       exists->(ind∈[b..e] and A(ind) and
          ∀i∈[b..ind-1]:(not A(i)))
```

shorting:

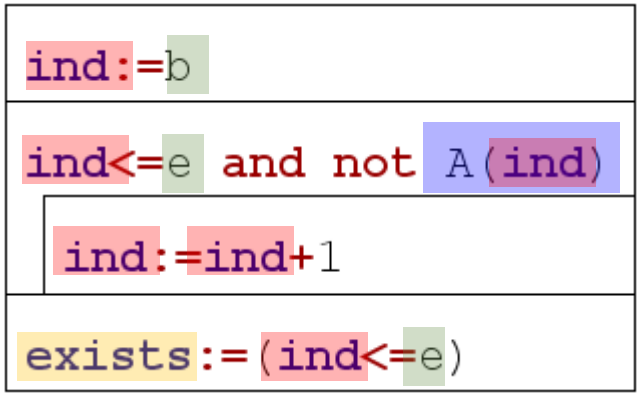
```
Post: (exists,ind)=SEARCH(i=b..e,A(i))
```

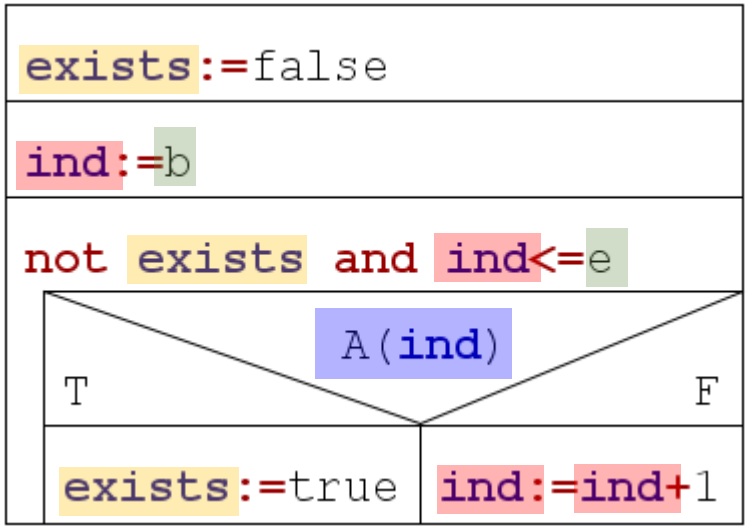
Algorithm:



Search

algorithm variations






```
Post:(exists,ind)=SEARCH(i=b..e,A(i))
```

Example

Let's define a non-1 and non-N divisor of the integer N!

Pattern:

In: b∈Z, e∈Z
Out: exists∈L, ind∈Z
Pre: -
Post:(exists,ind)=**SEARCH**(i=b..e,A(i))

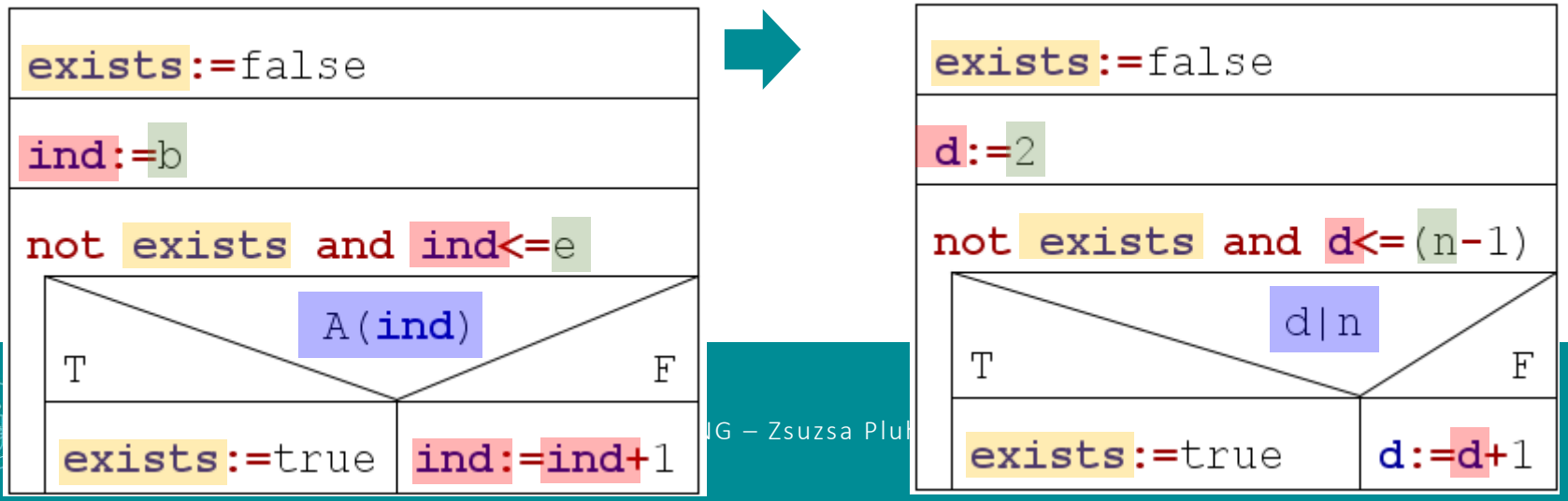
Specific problem:

In: n∈N
Out: d∈N, ex∈L
Pre: n>1
Post: (ex,d)=**SEARCH**(i=2..n-1, i|n)

Reduction:

exists	~	ex
ind	~	d
b..e	~	2..n-1
A(i)	~	i|n

Algorithm



Programming patterns
Decision

Decision

Tasks (examples):

1. Let's **decide** if an integer is prime or not!
2. Let's **tell if** a given word is the name of a month!
3. Based on the final marks of a student, **let's determine** if he/she fails the semester!
4. **Let's find if** a given word contains a vowel!
5. **Let's decide** if a sequence is monotonically increasing!
6. Based on the final marks, **let's determine** if the student is excellent (all marks are the best).

Decision

Tasks (examples):

1. Let's **decide** if an integer is prim
2. Let's **tell if** a given w
3. Based
4.
5. notonically
6. B marks, **let's determine** if the st ellent (all marks are the best).

What is common?

Let's determine if there is an item with given attribute in a sequence of "somethings"

A "narrowed" (output) version of search

Decision – example

algorithmic thinking

	mn	i<=12 and not= April	i<=12 and not= Apple
1	January	true	true
2	February	true	true
3	March	true	true
4	April	false	true
5	May		true
6	...		true
12	December		true
13			false

Task: Let's tell if a given word is the name of a month!

Specification:

```
In: name∈S, mn∈S[1..12]={"January","February",...}
Out: ismn∈L
Pre: n>1
Post:
ismn=∃i∈[1..n]:(mn[i]=name)
```

Algorithm:

```
i:=1
i<=12 and not (mn[i]=name)
    i:=i+1
ismn:=(i<=12)
```

Decision

Let an interval of integers [b..e] and a condition A: [b..e]→L be given.

Determine whether it exists a point at which A attains the "true" value on the interval [b..e]. (The empty interval is an equivalent of A evaluating to "false" on the whole interval.)

Specification:

In: b∈Z, e∈Z

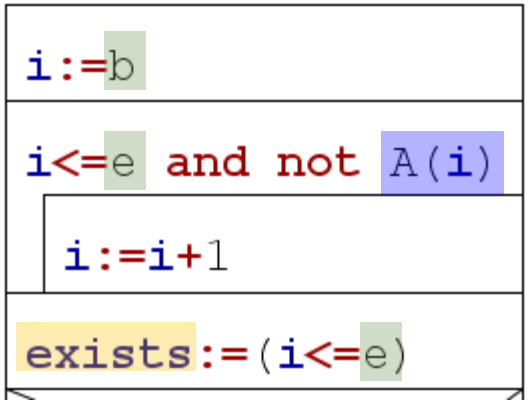
Out: exists∈L, ind∈Z

Pre: -

Post: exists=∃i∈[b..e]:(A(i))

shorting:

Post: exists=**EXISTS**(i=b..e,A(i))



Decision

Let an interval of integers [b..e] and a condition A: [b..e]→L be given.

Determine whether it exists a point at which A attains the "true" value on the interval [b..e]. (The empty interval is an equivalent of A evaluating to "false" on the whole interval.)

Specification:

In: b∈Z, e∈Z

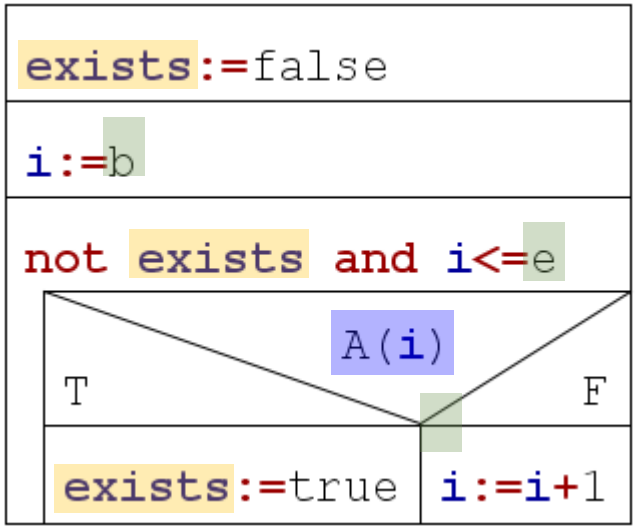
Out: exists∈L, ind∈Z

Pre: -

Post: exists=∃i∈[b..e]:(A(i))

shorting:

Post: exists=**EXISTS**(i=b..e,A(i))



Example

Based on the final marks of a student, let's determine if he/she fails the semester!

Pattern:

In: b∈Z, e∈Z
Out: exists∈L
Pre: -
Post:exists=**SEARCH**(i=b..e,A(i))

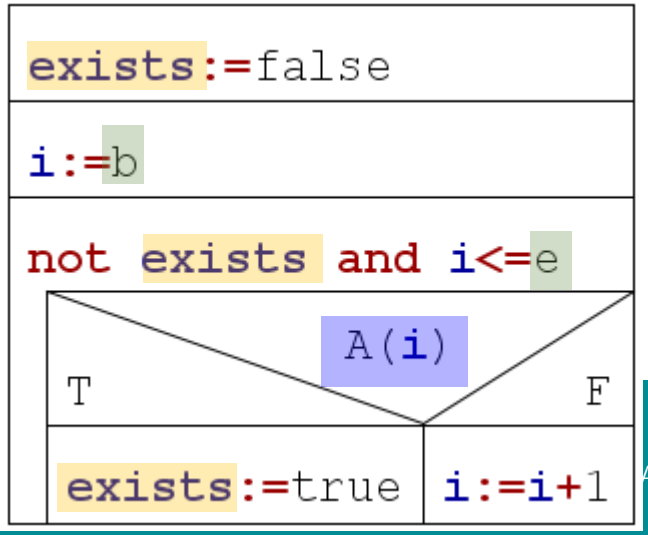
Specific problem:

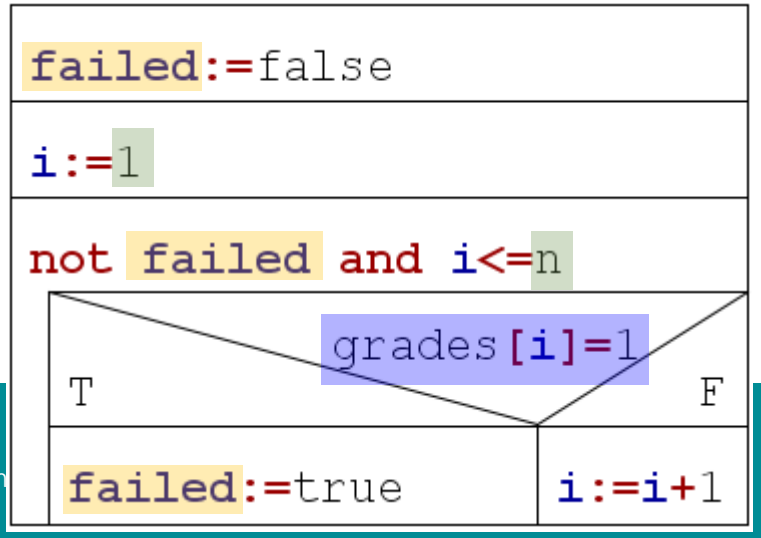
In: n∈N, grades∈N [1..n]
Out: failed∈L
Pre: ∀i∈[1..n]:(1<=grades[i]<=5)
Post: failed=**EXISTS**(i=1..n, grades[i]=1)

Reduction:

exists	~	failed
b..e	~	2..n-1
A(i)	~	grades[i]=1

Algorithm





Programming patterns
Selection

Selection

Tasks (examples):

1. We know the monthly income and expenses of a person. His assets grow by the end of the year. Let's **give** a month, **when** his asset grows!
2. Let's **define** the least divisor of a non-1 integer!
3. Let's **find** a vowel in an English word!
4. Let's **define** the serial number of a month given by its name!

Selection

Tasks (examples):

1. We know the
   perso
2.
3.
4. of a month given

What is common?

We have a sequence of "somethings", and we have to select **an** element that has a given attribute, and we know that at least one such element exists in the sequence.

A "narrowed" (input) version of search

Selection – example

algorithmic thinking

Task: Let's **define** the least divisor of a non-1 integer

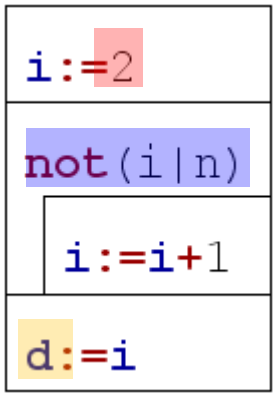
	i	i<=9?	i ∤ n?
	1		
e=	2	true	true
	3	true	false
	4		
	5		
	6		
	7		
	8		
n=	9		

	i	i<=5?	i ∤ n?
	1		
e=	2	true	true
	3	true	true
	4	true	true
n=	5	true	false

Specification:

```
In: n∈N
Out: d∈N
Pre: n>1
Post:
  1<d<=n and d|n and
  ∀i∈[2..d-1]:(i ∤ n)
```

Algorithm:



Selection

Let an interval of integers [b..e] and a condition A: [b..e]→L be given.

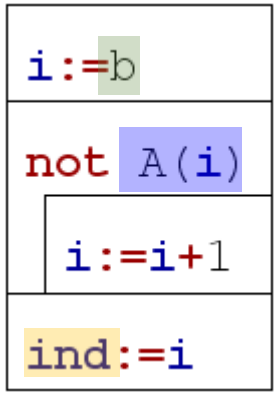
Determine the leftmost point starting with b at which A attains the "true" value on the interval [b..e] if we know that such point definitely exists

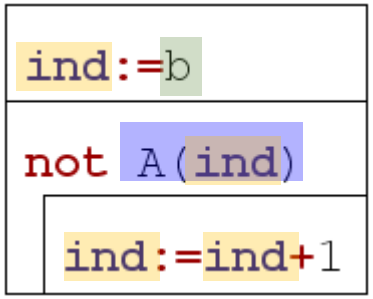
Specification:

In: b∈Z, e∈Z
Out: ind∈Z
Pre: ∃i∈[b..e]:(A(i))
Post: ind>=b and A(ind) and
∀i∈[b..ind-1]:(not A(i))

shorting:

Post: ind=**SELECT**(i=b..e,A(i))





Example

Let's find a vowel in an English word!

Pattern:

In: b∈Z, e∈Z
Out: ind∈Z
Pre: ∃i∈[b..e]:(A(i))
Post:ind=**SELECT**(i=b..e,A(i))

Specific problem:

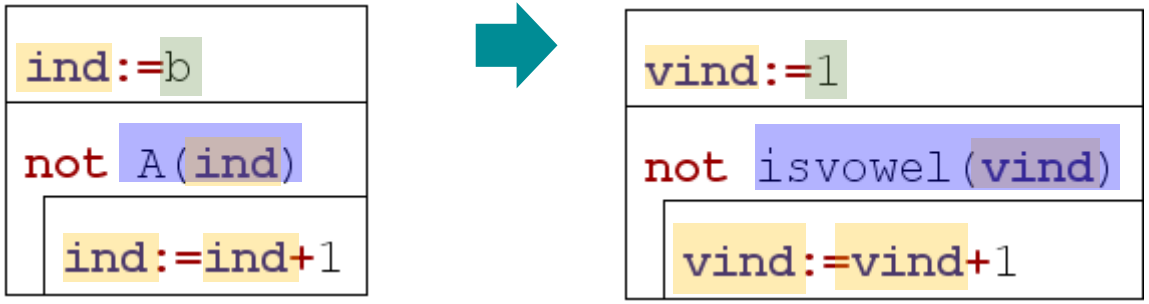
In: word∈S
Out: vind∈N
Pre: ∃i∈[1..length(word)]:(isvowel(word[i]))
Post: vind=**SELECT**(i=1..n, isvowel(word[i]))

Reduction:

ind	~	vind
b..e	~	1..n
A(i)	~	isvowel(word[i])

A attribute function

Algorithm



Example

Let's find a vowel in an English word!

Pattern:

In: b∈Z, e∈Z

Out: ind∈Z

Pre: ∃i∈[b..e]:(A(i))

Post:ind=**SELECT**(i=b..e,A(i))

Specific problem:

In: word∈S

Out: vind∈N

Func: isvowel: C->L

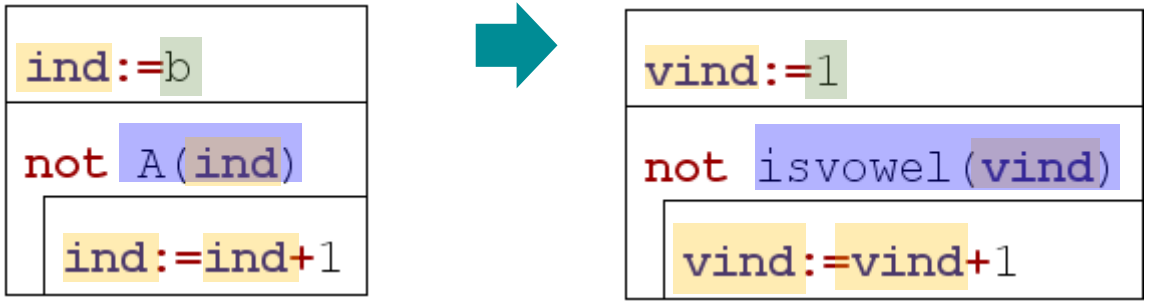
char

isvowel(ch)=(ch='a' or ch='e' or …)

Pre: ∃i∈[1..length(word)]:(isvowel(word[i]))

Post: vind=**SELECT**(i=1..n, isvowel(word[i]))

Algorithm



Programming patterns
Copy

Copy

Tasks (examples):

1. Let's define the absolute values of **all elements** in an array!
2. Let's convert **all letters** of a text to lowercase!
3. Let's calculate the sum of two vectors!
4. Let's calculate sin(x) values for a given x series!
5. We know the ordinal number of N months, let's give their names!

Copy

Tasks (examples):

1. Let's d… n

4. … x series!

5. …mber of N months, let's …

What is common?

We have a sequence of "somethings", and we have to assign another sequence to these. The type of the assigned values can be different from the original type of the elements, but the count remains the same, as well as the order of the elements in the sequence (mostly).

Copy – example

algorithmic thinking

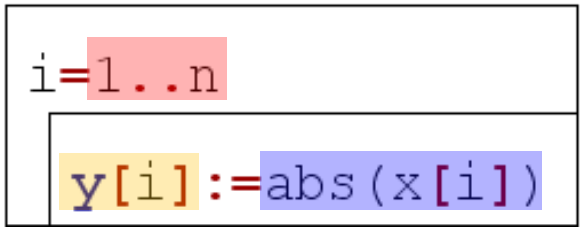
	x		y
1	3,3	→	3,3
2	-5,8	→	5,8
3	4,5	→	4,5
4	-2,2	→	2,2

Task: Let's define the absolute values of **all elements** in an array!

Specification:

```
In: n∈N, x∈R[1..n]
Out: y∈R[1..n]
Pre: -
Post: ∀i∈[1..n]:(y[i]=abs(x[i]))
```

Algorithm:



Copy

i		y
e	→	f(e)
e+1	→	f(e+1)
…	→	…
u	→	f(u)

Let an interval of integers [b..e] and a

function f: [b..e]→H be given.

Assign to every element (point) on the interval [b..e] the value imaged by the f function

Specification:

```
In: b∈Z, e∈Z
Out: y∈H[1..e-b+1]
Pre: -
Post: ∀i∈[b..e]:(y[i-b+1]=f(i))
```

shorting:

```
Post: y=COPY(i=b..e,f(i))
```

```
i=b..e
  y[i-b+1]:=f(i)
```

Example

Let's calculate the sum of two vectors!

Pattern:

In: b∈Z, e∈Z
Out: y∈H[1..u-e+1]
Pre: -
Post: y=**COPY**(i=b..e,f(i))

Specific problem:

In: n∈N, p∈R[1..n], q∈R[1..n]
Out: r∈R[1..n]
Pre: -
Post: r=**COPY**(i=1..n,p[i]+q[i])

Reduction:

y	~	r
b..e	~	1..n
f(i)	~	p[i]+q[i]

Algorithm

```
i=b..e
    y[i-b+1]:=f(i)
```

```
i=1..n
    r[i]:=p[i]+q[i]
```

Programming patterns
Multiple item selection

Multiple item selection

Tasks (examples):

1. Let's **define all** excellent students in a class!
2. Let's calculate the divisor**s** of an integer!
3. Let's **list all** the vowels from an English word!
4. Let's **collect all** the people who are taller than 180cm, from a set of people!
5. Let's **list** those days of the year when it didn't freeze at noon!

Multiple item selection

Tasks (examples):

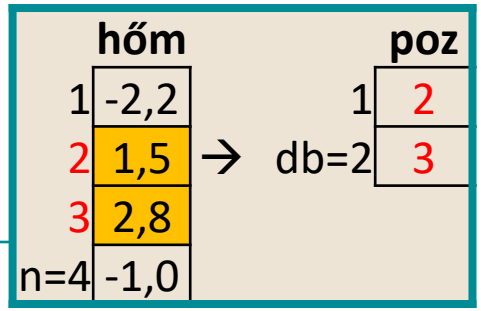
1. Let's **define all** exce
2. Let's c
3. glish word!
4. people who are taller than a set of people!
5. Let's **list** those days of the year when it didn't freeze at noon!

What is common?

We have to list **all** elements from a sequence of "somethings" which have a common attribute A.

MIS – example

algorithmic thinking



Task: Let's list those days of the year when it didn't freeze at noon!

Specification:

```
In: n∈N, temp∈R[1..n]
Out: cnt∈N, pos∈N[1..cnt]
Pre: ∀i∈[1..n]:(-100<=temp[i]<=100)
Post: cnt=COUNT(i=1..n,temp[i]>0) and
    ∀i∈[1..cnt]:(temp[pos[i]]>0) and
    ∀i∈[1..cnt]:(∀j∈[1..cnt]:(i<j->pos[i]<pos[j]))
```

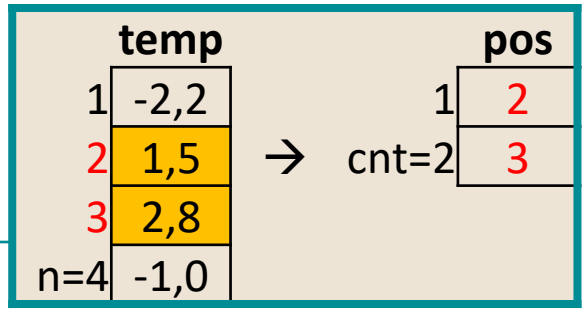
counting

The values of the selected elements are positive

No two indexes are the same in the pos array. Subset of the index range

MIS – example

algorithmic thinking



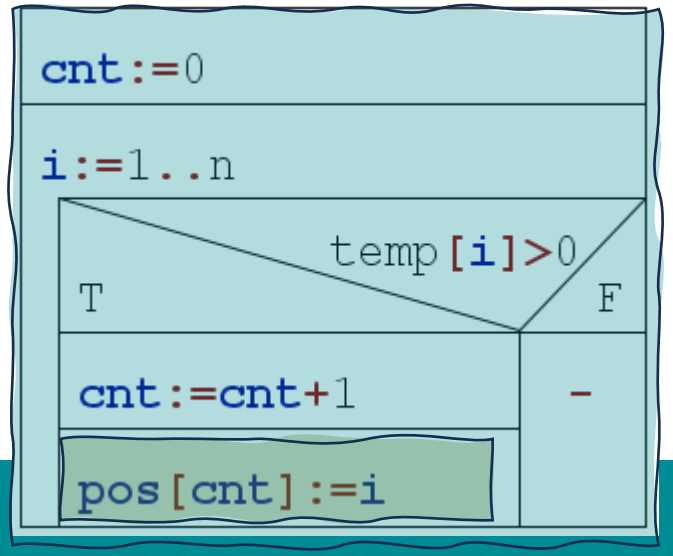
Task: Let's list those days of the year when it didn't freeze at noon!

Specification:

```
In: n∈N, temp∈R[1..n]
Out: cnt∈N, pos∈N[1..cnt]
Pre: ∀i∈[1..n]:(-100<=temp[i]<=100)
Post: cnt=COUNT(i=1..n,temp[i]>0) and
    ∀i∈[1..cnt]:(temp[pos[i]]>0) and
     poz⊆[1..n]
```

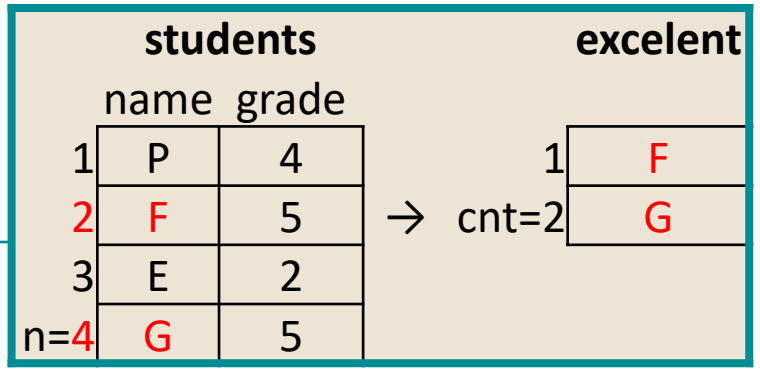
Subset of the index range

Algorithm:



MIS – example$_2$

algorithmic thinking



Task: Let's **define all** excellent students in a class! List the names…

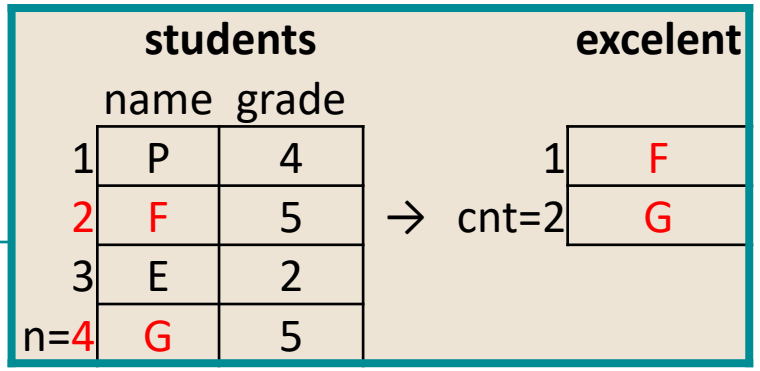
Specification:

```
In: n∈N, students∈Tstudent[1..n],
                    Tstudent=Name x Grade, Name=S, Grade=N
Out: cnt∈N, exs∈S[1..cnt]
Pre: ∀i∈[1..n]:(1<=students[i].grade<=5)
Post: cnt=COUNT(i=1..n,students[i].grade=5) and
  ∀i∈[1..cnt]:(∃j∈[1..n]:(
       students[j].grade=5 and exs[i]=students[j].name)) and
     ∀i∈[1..cnt]:(∀j∈[1..cnt]:(i<j->
       ∃ii∈[1..n]:(∃jj∈[1..n]:(
         ii<jj and
         students[ii].name=exs[i] and students[jj].name=exs[j]
       ))
     ))
```

same type as the field „names”

MIS – example$_2$

algorithmic thinking



Task: Let's **define all** excellent students in a class! List the names…

Specification:

```
In: n∈N, students∈Tstudent[1..n],
                    Tstudent=Name x Grade, Name=S, Grade=N
Out: cnt∈N, exs∈S[1..cnt]
Pre: ∀i∈[1..n]:(1<=students[i].grade<=5)
Post: cnt=COUNT(i=1..n,students[i].grade=5) and
  ∀i∈[1..cnt]:(∃j∈[1..n]:(
      students[j].grade=5 and exs[i]=students[j].name)) and
    ∀i∈[1..cnt]:(∀j∈[1..cnt]:(i<j->
      ∃ii∈[1..n]:(∃jj∈[1..n]:(
        ii<jj and
        students[ii].name=exs[i] and students[jj].name=exs[j]
      ))
    ))
```

counting

the selected names are assigned to a grade 5

The selected names are assigned to different indexes. Subset of the index range

MIS – example$_2$

algorithmic thinking

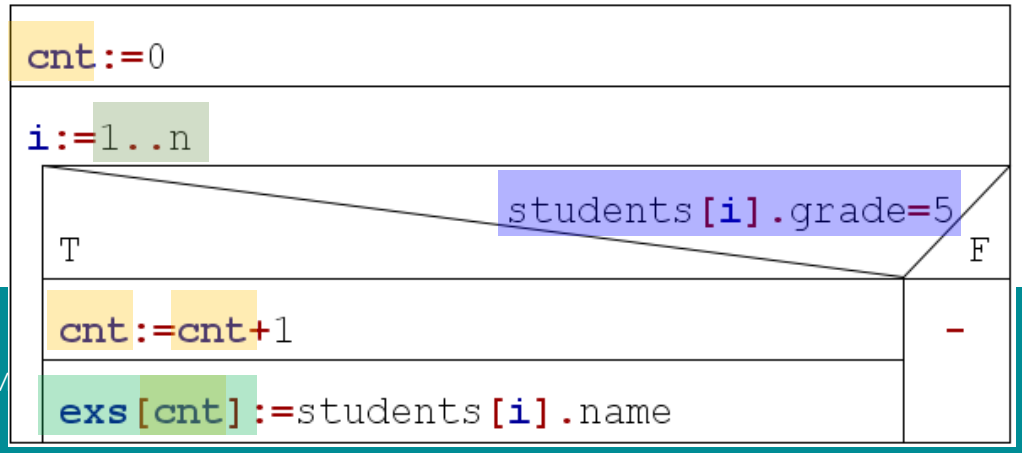
	students			excelent	
	name	grade			
1	P	4		1	F
2	F	5	→	cnt=2	G
3	E	2			
n=4	G	5			

Task: Let's **define all** excellent students in a class! List the names…

Specification:

```
In: n∈N, students∈Tstudent[1..n],
                    Tstudent=Name x Grade, Name=S, Grade=N
Out: cnt∈N, exs∈S[1..cnt]
Pre: ∀i∈[1..n]:(1<=students[i].grade<=5)
Post: cnt=COUNT(i=1..n,students[i].grade=5) and
   ∀i∈[1..cnt]:(∃j∈[1..n]:(
        students[j].grade=5 and exs[i]=students[j].name)) and
     exs⊆students.name
```

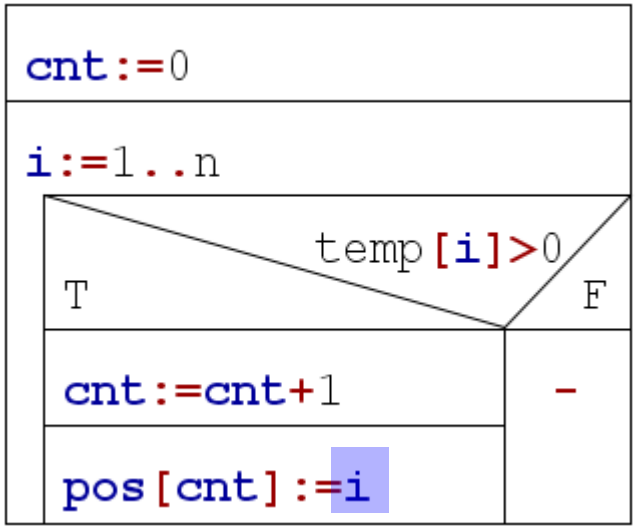
Subset of the index range

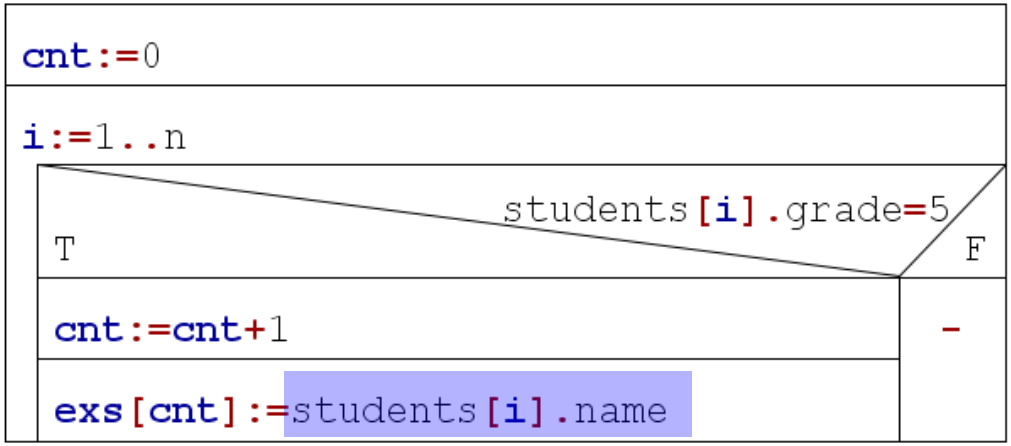


MIS conclusion

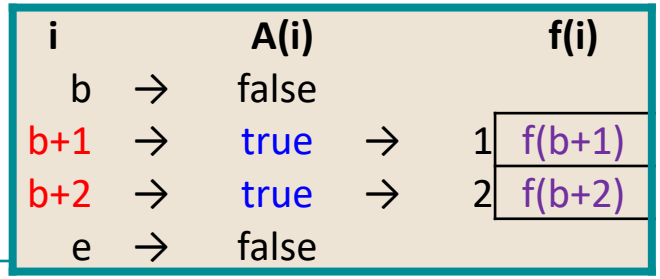
The result can be

- the elements of the interval (index)
- the values assigned to the elements of the interval





Multiple item selection



Let an interval of integers [b..e], a condition A: [b..e]→L and a function f: [b..e]→H be given.

Determine the f values of the point(s) at which A attains the "true" value on the interval [b..e]

Specification:

In: b∈Z, e∈Z

Out: cnt∈N, y∈H[1..cnt]

Pre: -

Post: cnt=**COUNT**(i=b..e,A(i)) and
∀i∈[1..cnt]:(∃j∈[b..e]:A(j) and y[i]=f(j))
and y⊆(f(b),f(b+1),...,f(e))

Multiple item selection

i		A(i)			f(i)
b	→	false			
b+1	→	true	→	1	f(b+1)
b+2	→	true	→	2	f(b+2)
e	→	false			

Let an interval of integers [b..e], a condition A: [b..e]→L and a function f: [b..e]→H be given.

Determine the f values of the point(s) at which A attains the "true" value on the interval [b..e]

Specification:

```
In: b∈Z, e∈Z
Out: cnt∈N, y∈H[1..cnt]
Pre: -
Post: cnt=COUNT(i=b..e,A(i)) and
    ∀i∈[1..cnt]:(∃j∈[b..e]:A(j) and y[i]=f(j))
 and y⊆(f(b),f(b+1),...,f(e))
```

f and A assign to the same element!

shorting:

```
Post: (cnt,y)=MIS(i=b..e,A(i),f(i))
```

Multiple item selection

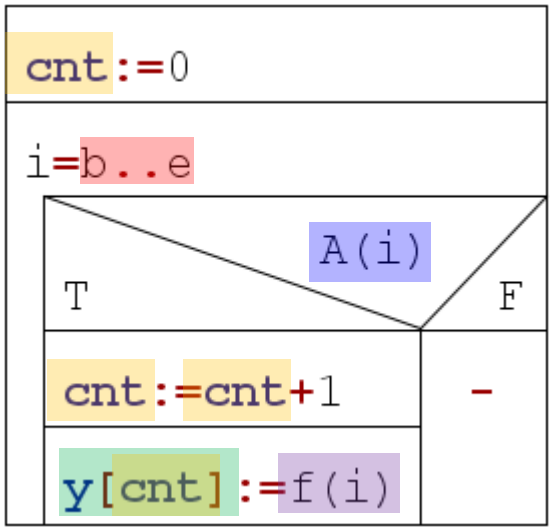
i		A(i)		f(i)
b	→	false		
b+1	→	true	→	1 f(b+1)
b+2	→	true	→	2 f(b+2)
e	→	false		

Let an interval of integers [b..e], a condition A: [b..e]→L and a function f: [b..e]→H be given.

Determine the f values of the point(s) at which A attains the "true" value on the interval [b..e]

Specification:

```
In: b∈Z, e∈Z
Out: cnt∈N, y∈H[1..cnt]
Pre: -
Post: (cnt,y)=MIS(i=b..e,A(i),f(i))
```



Example

Let's calculate the divisors of an integer!

Pattern:

In: b∈Z, e∈Z

Out: cnt∈N, y∈H[1..cnt]

Pre: -

Post: (cnt,y)=**MIS**(i=b..e,A(i),f(i))

Specific problem:

In: n∈N

Out: cnt∈N, divs∈N[1..cnt]

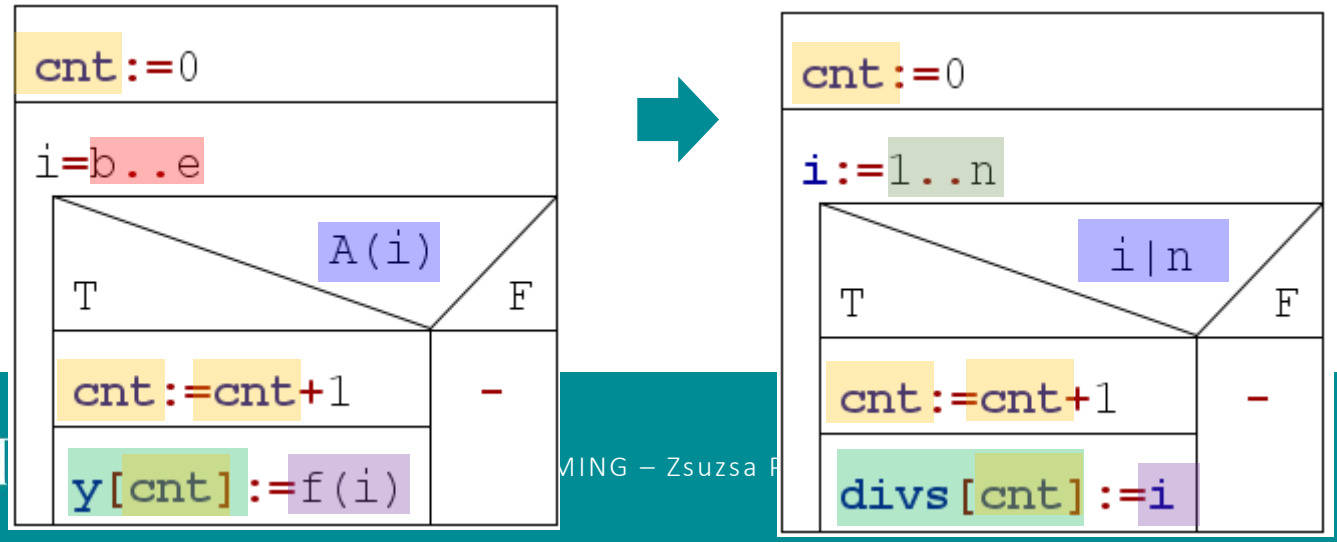
Pre: -

Post: (cnt,divs)=**MIS**(i=1..n,i|n,i)

Reduction:

cnt, y	~	cnt, divs
b..e	~	1..n
A(i)		i|n
f(i)	~	i

Algorithm



Programming patterns **conclusions**

Programming patterns

1. Summation
2. Counting
3. Maximum selection
4. Copy
5. Multiple item selection

problems with sum and "all"
↓
counting loop

6. Search
7. Decision
8. Selection

problem with "exists "
↓
conditional loop

Programming patterns

1. Summation
2. Counting

on empty interval

3. Maximum selection

at least 1 element in the interval

4. Copy
5. Multiple item selection
6. Search
7. Decision

on empty interval

8. Selection

at least 1 element in the interval

Programming patterns

1. Summation
2. Counting
3. Maximum selection

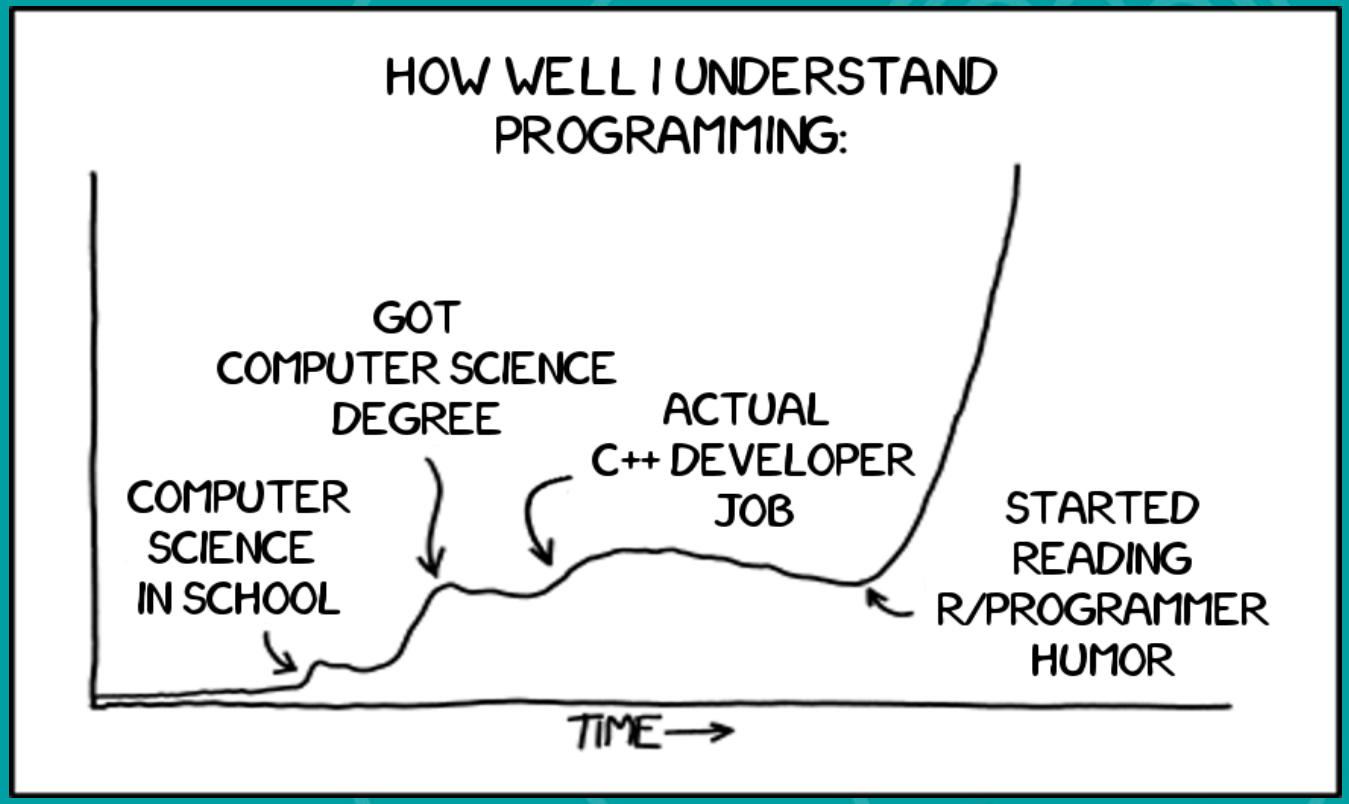
output: single value(s)

4. Copy
5. Multiple item selection

output: sequence

6. Search
7. Decision
8. Selection

output: singe value(s)



Thank you for your attention!